IRIS FLOWER CLASSIFICATION

Python • Git & GitHub • Automated Testing • CI/CD

Project Report

Jashan Shah

J054

1. Introduction

This project demonstrates a small machine-learning workflow for classifying Iris flower species with a Decision Tree classifier. It combines Python project setup, dependency management with uv, model execution, pytest-based automated tests, Git version control, GitHub repository management, and a GitHub Actions continuous-integration workflow.

2. Tools and Technologies

Python, scikit-learn, pytest, uv, Git, GitHub, GitHub Actions, and GitHub Desktop.

3. Project Setup and Dependencies

The project directory and source/test folders are created first. The project is initialized with uv, and scikit-learn and pytest are added as dependencies. The virtual environment and lock file support repeatable setup.

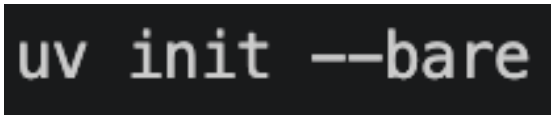


Figure 1. Initializing the project with uv.



Figure 2. Creating the README and .gitignore files.



Figure 3. Creating the source, tests, and documentation directories.

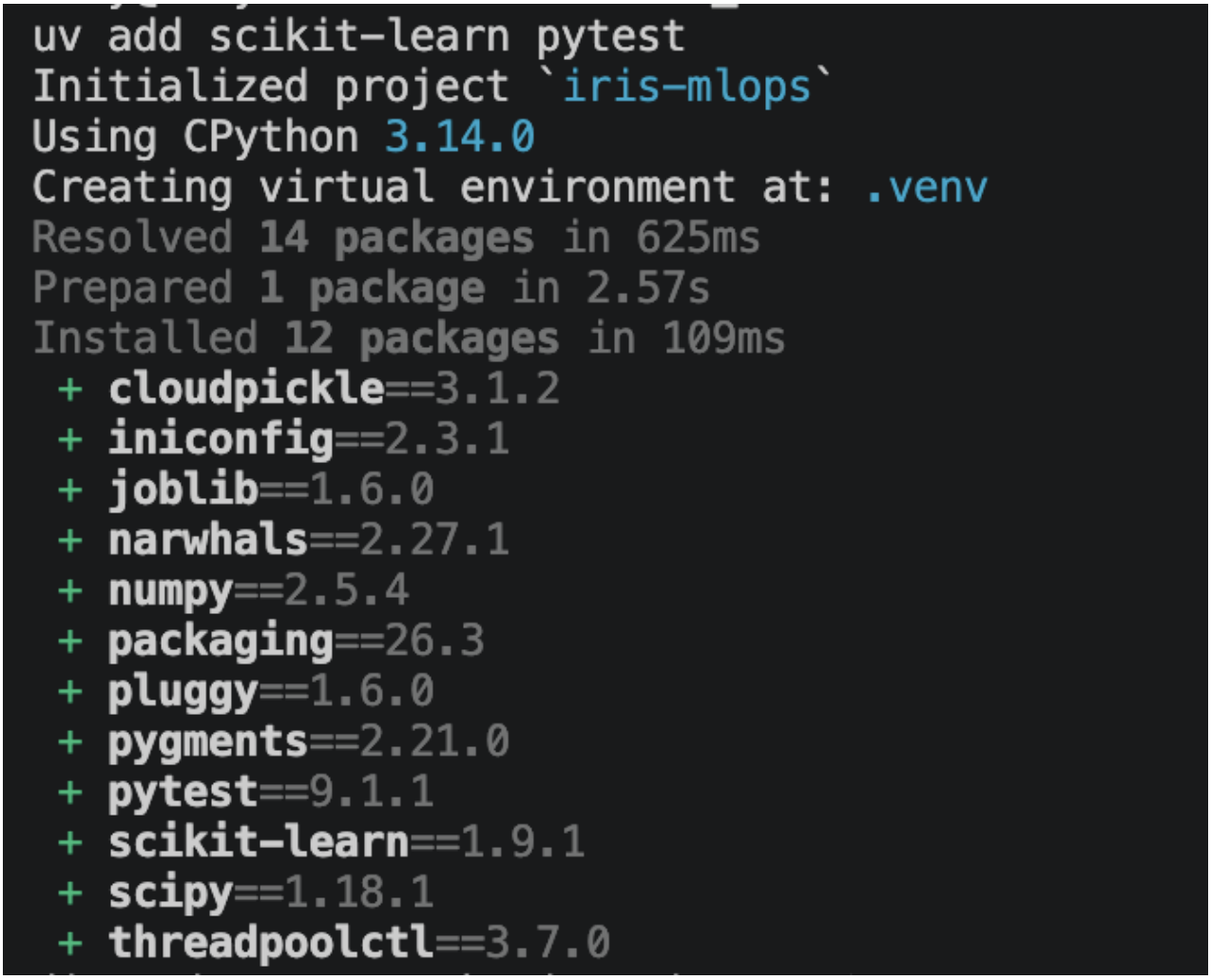


Figure 4. Installing scikit-learn and pytest and resolving dependencies.

4. Model Implementation and Execution

The model loads the built-in Iris dataset, splits it into training and test sets, trains a Decision Tree classifier, and evaluates predictions using accuracy. The captured run reports an accuracy of 0.93 on 30 test samples.

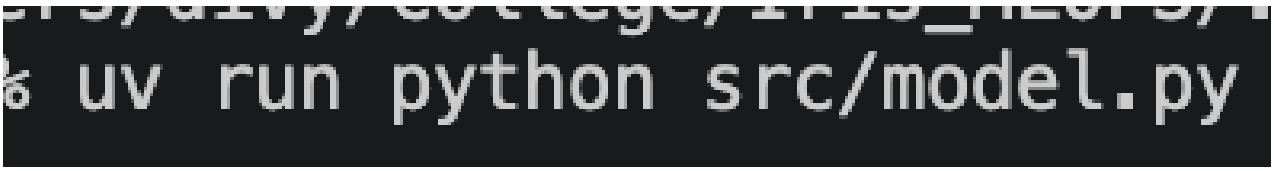


Figure 5. Running the model script.

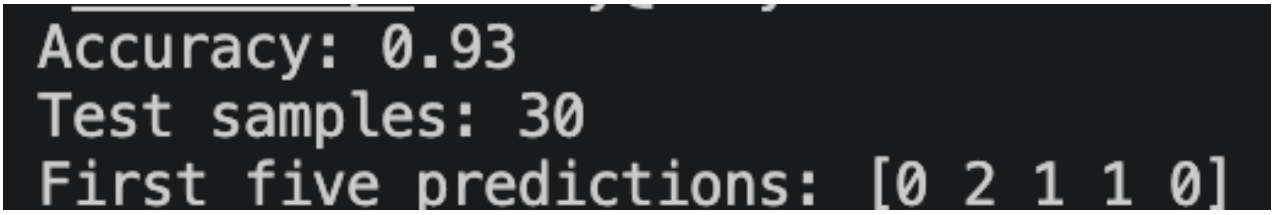


Figure 6. Model output: accuracy 0.93, 30 test samples, and sample predictions.

5. Automated Testing

The pytest suite contains four tests covering successful training, test-data dimensions, valid class labels, and model accuracy/predictions. The screenshots show all four tests passing.

```
========================================= test session starts =========================================
platform darwin -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0 -- /Users/divy/College/Iris_MLOPS/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/divy/College/Iris_MLOPS
configfile: pyproject.toml
collected 4 items

tests/test_model.py::test_model_trains_successfully PASSED                                      [ 25%]
tests/test_model.py::test_test_dataset_dimensions PASSED                                        [ 50%]
tests/test_model.py::test_predictions_have_valid_labels PASSED                                  [ 75%]
tests/test_model.py::test_model_accuracy PASSED                                                 [100%]

========================================== 4 passed in 0.94s ==========================================
```

Figure 7. All four automated tests pass.

```
uv run python -m pytest -v --junitxml=pytest-report.xml
Resolved 14 packages in 23ms
Checked 12 packages in 5ms
========================================= test session starts =========================================
platform darwin -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0 -- /Users/divy/College/Iris_MLOPS/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/divy/College/Iris_MLOPS
configfile: pyproject.toml
collected 4 items

tests/test_model.py::test_model_trains_successfully PASSED                                      [ 25%]
tests/test_model.py::test_test_dataset_dimensions PASSED                                        [ 50%]
tests/test_model.py::test_predictions_have_valid_labels PASSED                                  [ 75%]
tests/test_model.py::test_model_accuracy PASSED                                                 [100%]

---------------------- generated xml file: /Users/divy/College/Iris_MLOPS/pytest-report.xml ----------------------
========================================== 4 passed in 0.88s ==========================================
```

Figure 8. Pytest output and the generated JUnit XML test report.

6. Git Version Control and Initial Commit

Git is initialized on the main branch. The project files are reviewed and staged, then committed as the initial project snapshot. The screenshots include the repository status and commit summary.

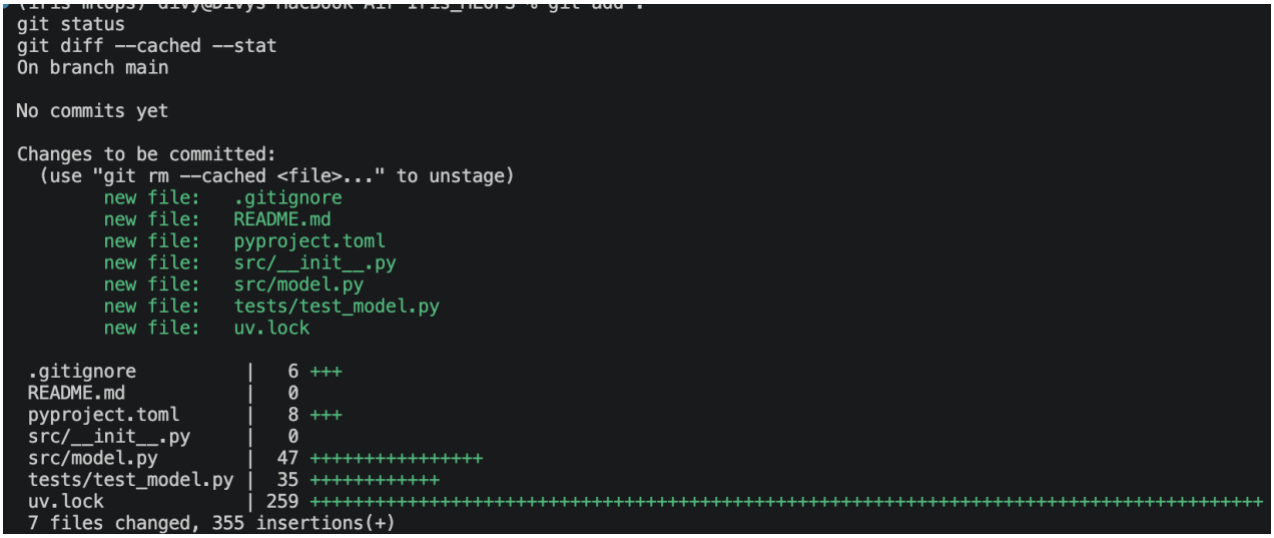
```
git branch -M main
git status
Initialized empty Git repository in /Users/divy/College/Iris_MLOPS/.git/
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        pyproject.toml
        src/
        tests/
        uv.lock

nothing added to commit but untracked files present (use "git add" to track)
```

Figure 9. Initial repository status before files are committed.


git status
git diff --cached --stat
On branch main
No commits yet
Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file: .gitignore
new file: README.md
new file: pyproject.toml
new file: src/__init__.py
new file: src/model.py
new file: tests/test_model.py
new file: uv.lock
.gitignore | 6 +++
README.md | 0
pyproject.toml | 8 +++
src/__init__.py | 0
src/model.py | 47 +++++++++++++++++
tests/test_model.py | 35 +++++++++++
uv.lock | 259 ++++++++++++++++

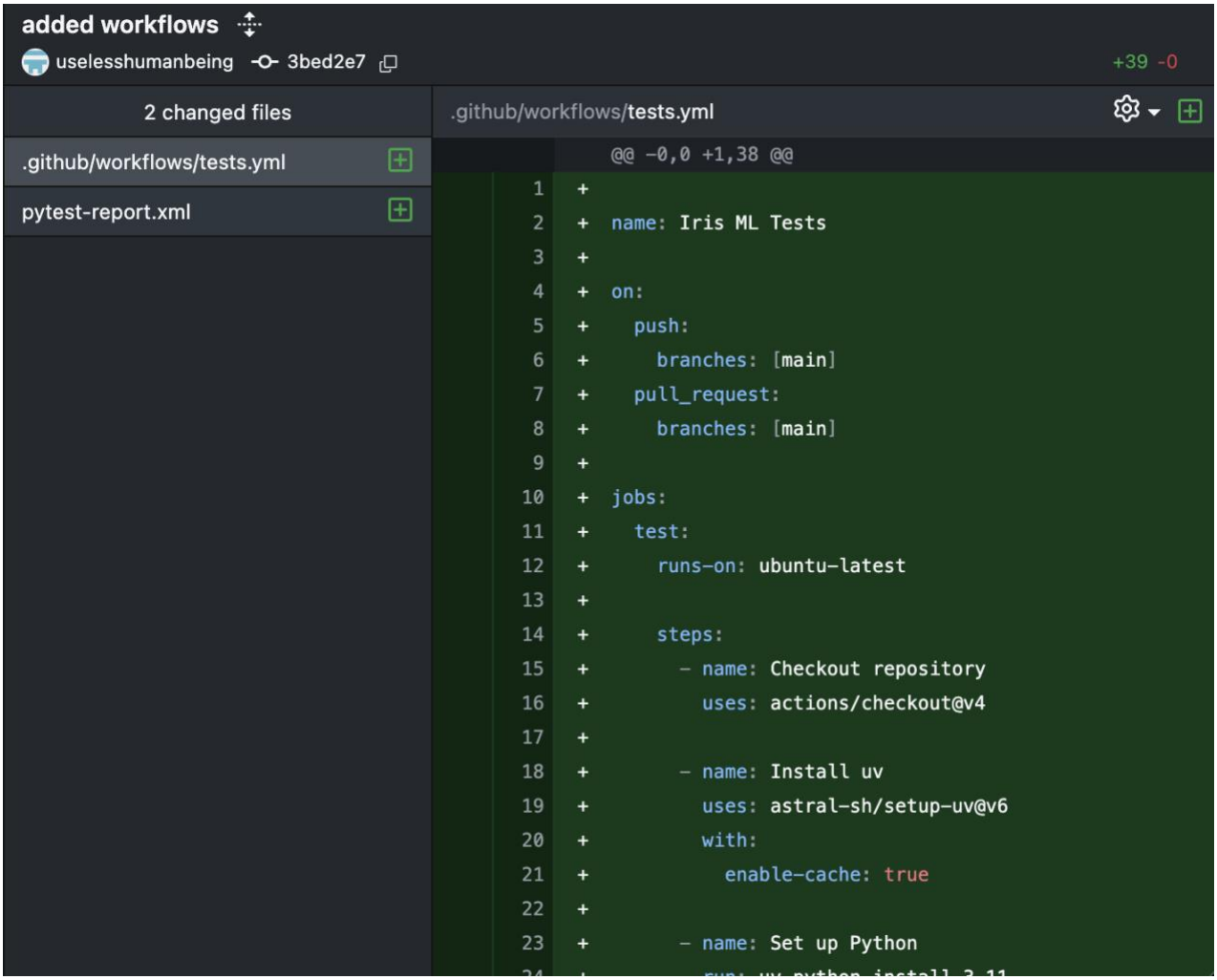


Figure 12. Adding the GitHub Actions workflow and test-report artifact configuration.

A temporary change that removed the model code caused the workflow to fail. Restoring the model implementation allowed the workflow to pass again. This demonstrates that the CI workflow detects a broken project and validates the corrected version.

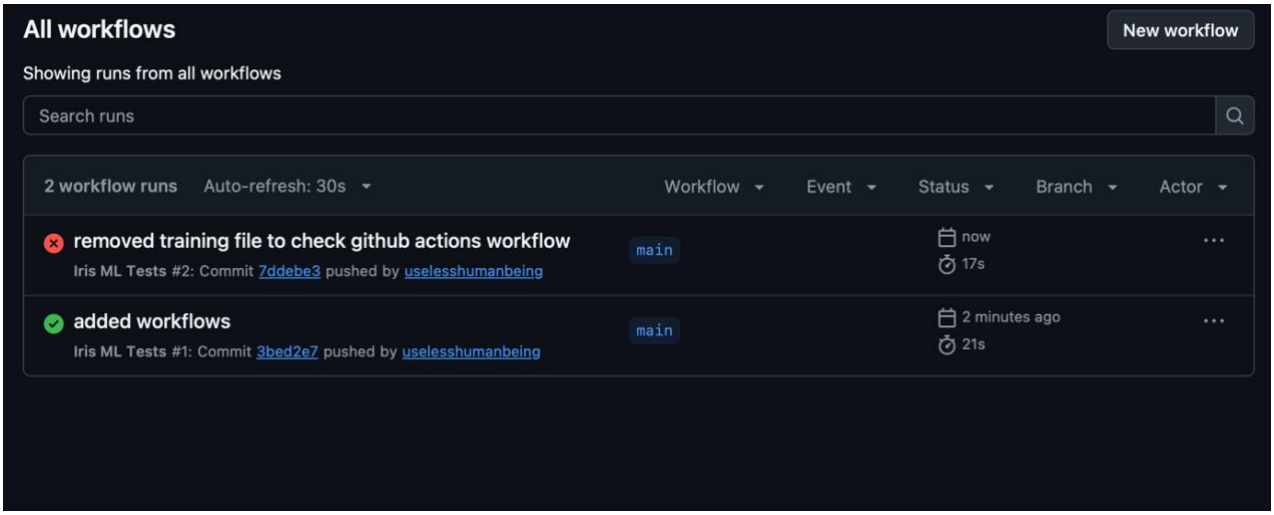


Figure 13. Workflow history showing the failed run after a temporary breaking change and a later successful run.

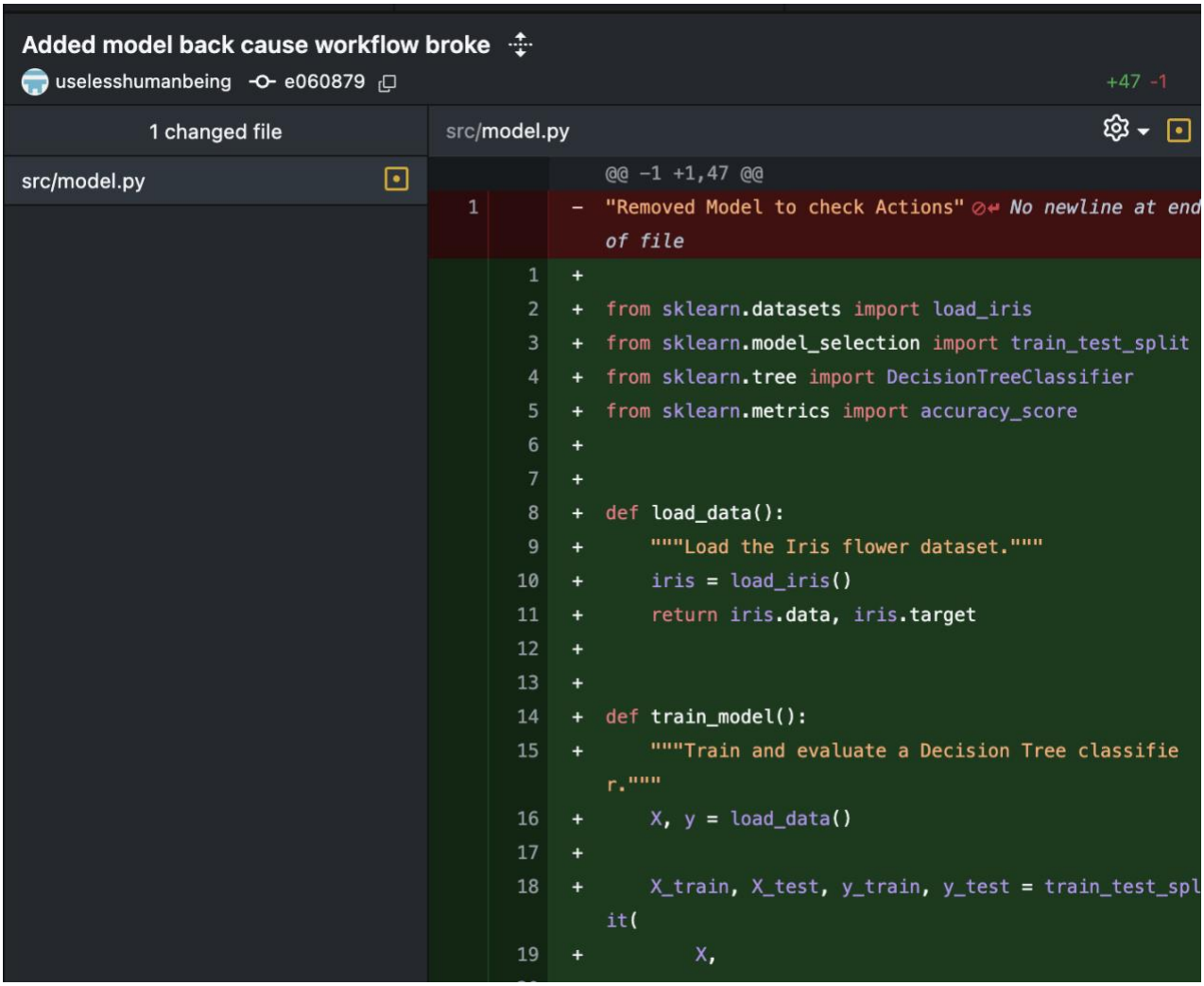


Figure 14. Restoring the model implementation in the repository.

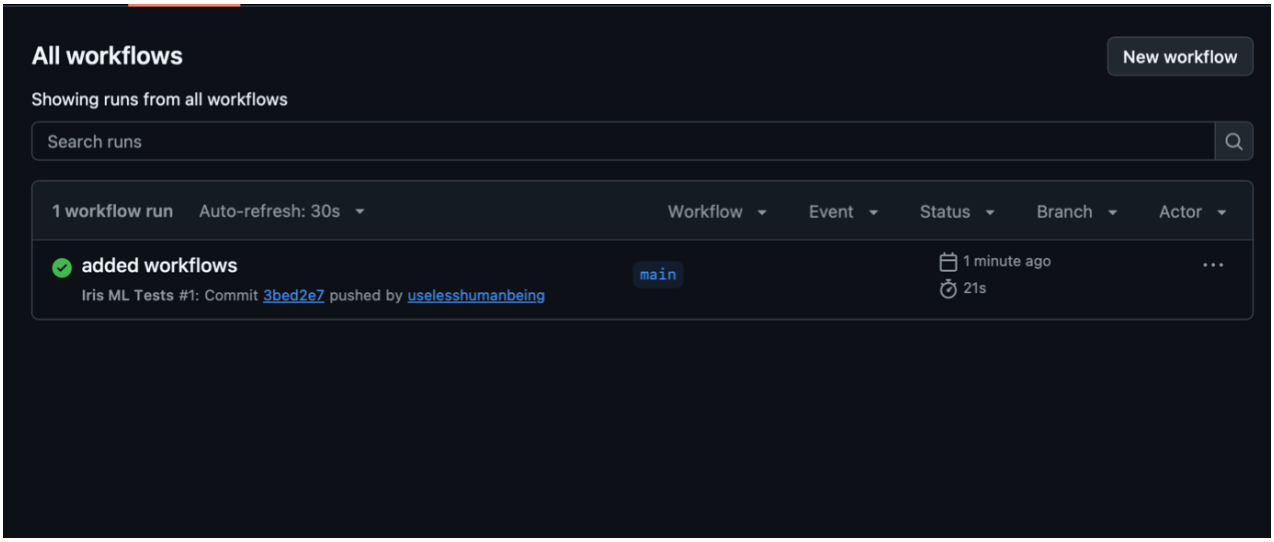


Figure 15. GitHub Actions showing a successful workflow run.

8. Documentation and Repository History

The README documents the project overview, technologies, directory structure, setup, execution, testing, and CI workflow. GitHub Desktop is used to review the repository history and tracked files.

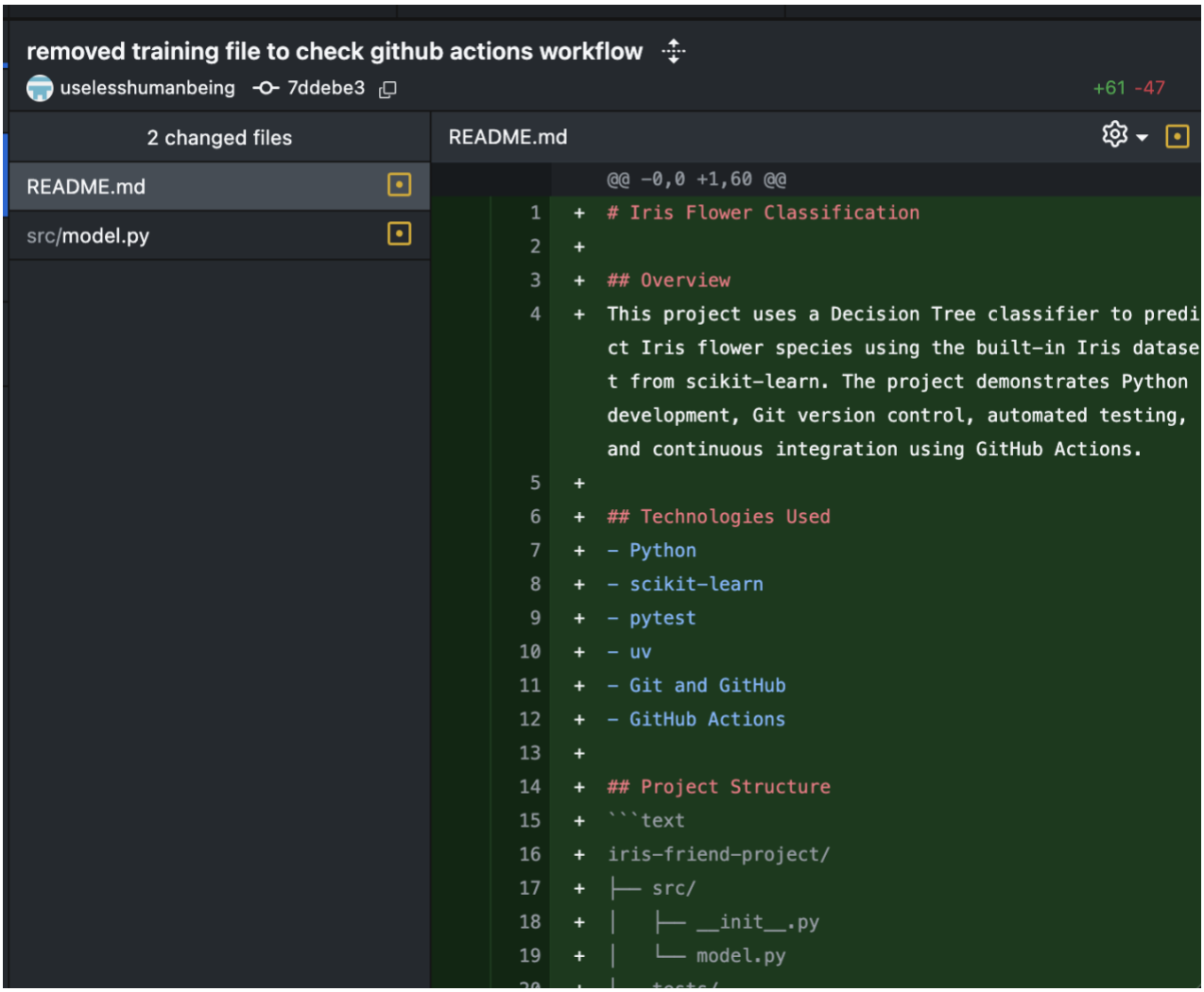


Figure 16. README documentation update and successful workflow status.

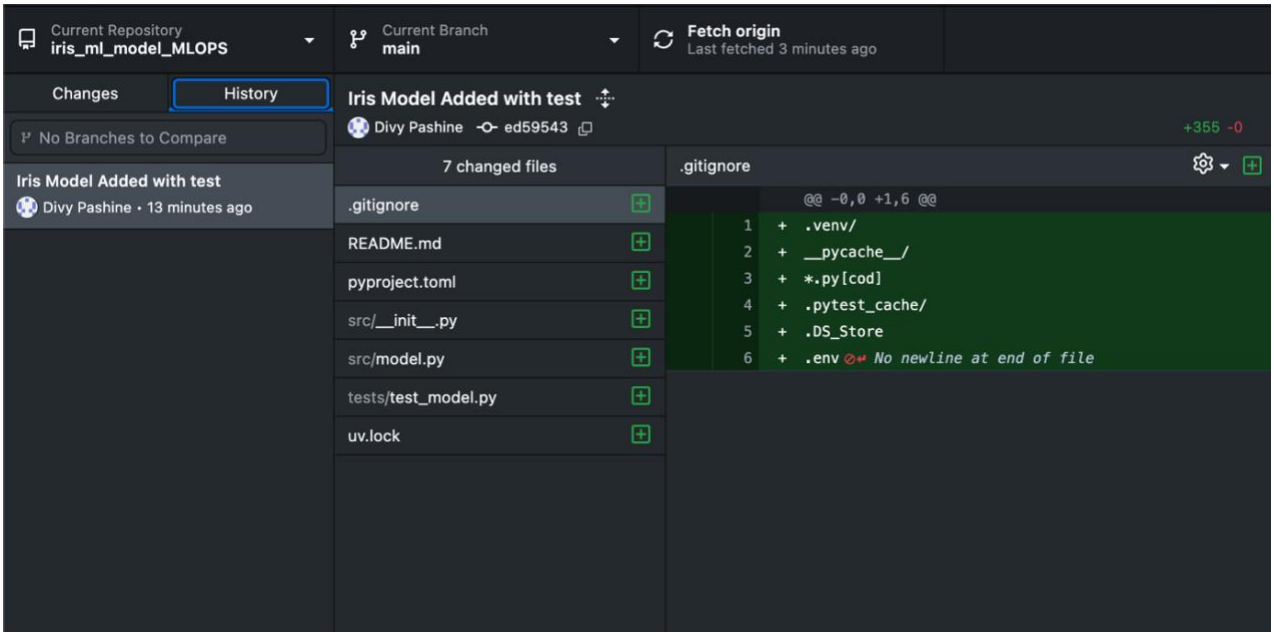


Figure 17. GitHub Desktop repository history and project files.

9. Results

The captured model run achieved 0.93 accuracy on the 30-sample test set. All four pytest tests passed in the recorded run, and the final GitHub Actions screenshot shows a successful CI run. The workflow also produces a JUnit XML report artifact when the workflow reaches the test/report steps.

10. Conclusion

The project demonstrates the end-to-end workflow for organizing a Python ML project, managing dependencies, validating behavior with automated tests, tracking changes with Git, documenting the project, and running tests automatically through GitHub Actions.

Note: The terminal screenshots visibly contain the local path/name “Iris_MLOPS” and some GitHub screenshots show a different account/repository naming context. The report orders and explains the supplied evidence without altering the screenshots. If this report is intended to document Jashan Shah’s separate repository specifically, replace any screenshots that belong to the earlier/original repository with matching screenshots from Jashan’s own repository.